231501046
GANESH S AD23633

EXPERIMENT 3
Generative Adversarial Network (GAN) for Synthetic Image Generation

---

AIM:

To implement a Generative Adversarial Network (GAN) to generate synthetic handwritten digit images using the MNIST dataset with PyTorch.

PROCEDURE:

1. Install required libraries: torch, torchvision, matplotlib.
2. Set device to CUDA if available, else CPU.
3. Define hyperparameters: latent_dim=100, batch_size=64, epochs=60, lr=0.0001.
4. Load MNIST dataset with normalization transform (mean=0.5, std=0.5) and create DataLoader.
5. Define Generator network: 4 Linear layers (100→256→512→1024→784) with ReLU activations and Tanh output.
6. Define Discriminator network: 3 Linear layers (784→512→256→1) with LeakyReLU(0.2) activations and Sigmoid output.
7. Initialize both networks and move to device.
8. Use BCELoss as criterion; Adam optimizers (lr=0.0001) for both Generator and Discriminator.
9. Training loop: For each epoch, for each batch of real images:
   a. Train Discriminator on real images (label=1) and fake images (label=0), compute d_loss.
   b. Train Generator: generate fake images, pass through Discriminator with real labels, compute g_loss.
10. Print D Loss and G Loss every epoch.
11. After training, generate 16 sample images from random noise and display in a 4x4 grid.

CODE:

```
import torch, torch.nn as nn, torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
latent_dim=100; batch_size=64; epochs=60; lr=0.0001

transform = transforms.Compose([transforms.ToTensor(),
                                transforms.Normalize((0.5,),(0.5,))])
mnist = datasets.MNIST(root='./data',train=True,transform=transform,download=True)
loader = DataLoader(mnist, batch_size=batch_size, shuffle=True)

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim,256), nn.ReLU(True),
            nn.Linear(256,512), nn.ReLU(True),
            nn.Linear(512,1024), nn.ReLU(True),
            nn.Linear(1024,28*28), nn.Tanh())
    def forward(self,z): return self.net(z)

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(28*28,512), nn.LeakyReLU(0.2,inplace=True),
            nn.Linear(512,256), nn.LeakyReLU(0.2,inplace=True),
```

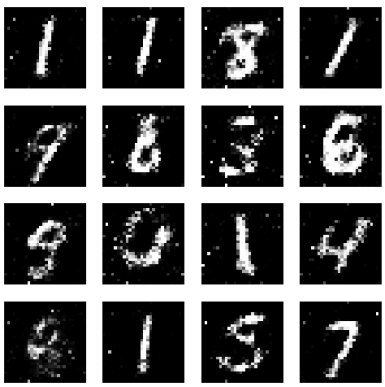
```
            nn.Linear(256,1), nn.Sigmoid())
    def forward(self,x): return self.net(x)

G=Generator().to(device); D=Discriminator().to(device)
criterion=nn.BCELoss()
optimizer_G=optim.Adam(G.parameters(),lr=lr)
optimizer_D=optim.Adam(D.parameters(),lr=lr)

for epoch in range(epochs):
    for real_imgs,_ in loader:
        real_imgs=real_imgs.view(-1,28*28).to(device)
        real_labels=torch.ones(real_imgs.size(0),1).to(device)
        fake_labels=torch.zeros(real_imgs.size(0),1).to(device)
        optimizer_D.zero_grad()
        loss_real=criterion(D(real_imgs),real_labels)
        z=torch.randn(real_imgs.size(0),latent_dim).to(device)
        fake_imgs=G(z)
        loss_fake=criterion(D(fake_imgs.detach()),fake_labels)
        d_loss=loss_real+loss_fake; d_loss.backward(); optimizer_D.step()
        optimizer_G.zero_grad()
        outputs=D(fake_imgs)
        g_loss=criterion(outputs,real_labels)
        g_loss.backward(); optimizer_G.step()
    print(f'Epoch [{epoch+1}/{epochs}] D:{d_loss.item():.4f} G:{g_loss.item():.4f}')

z=torch.randn(16,latent_dim).to(device)
fake_images=G(z).view(-1,28,28).detach().cpu()
plt.figure(figsize=(6,6))
for i in range(16):
    plt.subplot(4,4,i+1); plt.imshow(fake_images[i],cmap='gray'); plt.axis('off')
plt.show()
```

OUTPUT:



231501046
GANESH S AD23633

RESULT:

The GAN was successfully implemented and trained for 60 epochs on the MNIST dataset. The Generator learned to produce realistic handwritten digit images while the Discriminator learned to distinguish real from fake images. By epoch 60, the Generator Loss stabilized and the generated images visually resembled authentic MNIST digits (0-9), demonstrating the effectiveness of adversarial training for synthetic image generation.